Kadence

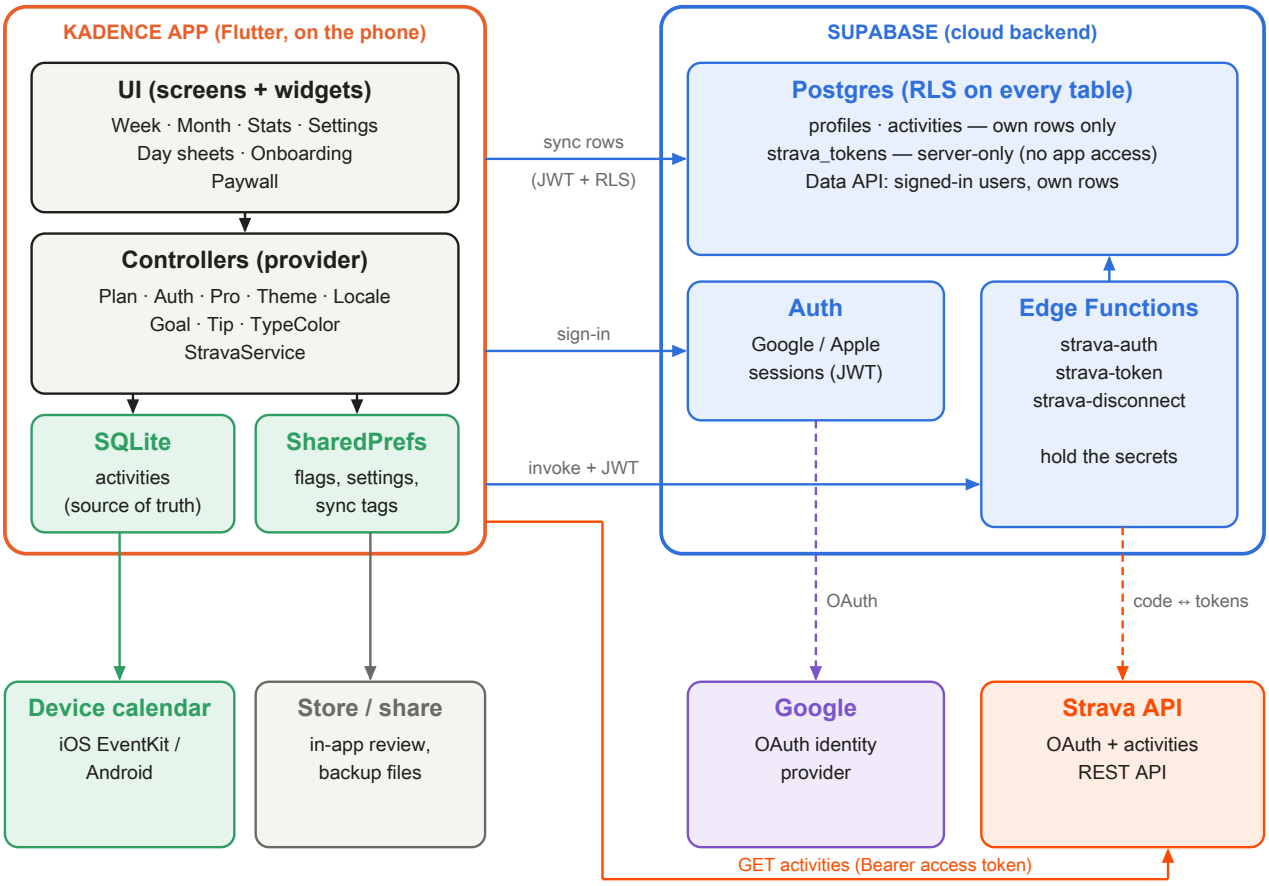
Architecture overview

App structure · local data · authentication (Google + Supabase)
cloud sync · Strava integration · secrets · Pro gating

Snapshot of branch dev · October 2026

1. The big picture

Kadence is a Flutter app for planning a week of sport. It is **offline-first**: everything the user does is written to a SQLite database on the phone, and the app is fully usable with no account and no network. The cloud is an optional extra for **Pro** users: signing in turns on backup and cross-device sync through Supabase, and unlocks the Strava import.



Solid = data flow · Dashed = server-side handshakes · The landing page (Vercel) is separate static HTML

Who does what

Piece	Role	Runs where
Flutter app	All UI, all business logic, local storage, calls to Supabase and Strava	User's phone
Supabase Auth	Signs users in through Google/Apple and issues session tokens (JWTs)	Supabase cloud
Supabase Postgres	Cloud copy of activities + user profiles + Strava tokens. Row Level Security keeps each user to their own rows	Supabase cloud
Edge Functions	Small TypeScript programs for anything that needs a secret (Strava client secret, refresh tokens)	Supabase cloud (Deno)
Google	Identity provider: proves who the user is	Google
Strava API	Source of completed activities (read-only for Kadence)	Strava
Landing page	Static marketing site + privacy policy	Vercel

2. Inside the app

The code follows a simple layered layout. Screens never talk to the database or network directly — they read and call **controllers**, which own the state and do the I/O.

```
lib/
  main.dart        start-up: Supabase, prefs, DB, controllers, deep links
  app.dart         MaterialApp + theme + onboarding gate
  models/          Activity, ActivityType, DayStatus
  state/           controllers + services (the "brain")
  screens/         one folder per screen, widgets/ subfolders
  widgets/         shared k_* building blocks
  theme/           colour, spacing, text tokens
  l10n/            translations (en, pt, pt-BR, es, fr)
supabase/
  migrations/      SQL for cloud tables + security rules
  functions/       edge functions (server code)
```

State management

The app uses **provider** with one `ChangeNotifier` per domain, all created in `main.dart`. A screen calls `context.watch<PlanController>()` to rebuild when data changes, or `context.read` to call an action.

Controller / service	Owns
PlanController	All activities in memory (date → list). Every write goes to SQLite, and to the cloud when signed in
AuthController	Supabase session; runs the full sync on sign-in
ProController	Whether the user is Pro (local flag for now); forces sign-out on downgrade
StravaService	Strava connect / sync / disconnect
SyncService, ActivityDb, BackupService, CalendarService, ReviewService	Helpers: cloud rows, SQLite, JSON backup, device calendar, store review prompt
Theme, Locale, Goal, Tip, TypeColor, Onboarding	Small settings, each persisted in SharedPreferences

Data model and local storage

An **Activity** has a date, a status (planned / today / done), a name, a type (+ optional sub-type), optional time, duration and notes, and an optional link to a calendar event. Activities live in one SQLite table on the phone, which is the **source of truth**. Settings and small flags live in SharedPreferences (key-value storage).

Activity IDs carry their origin, which matters for the cloud table where IDs must be unique across all users:

Prefix	Created by	Example
a	User adds a session in the app (counter)	`a42`
i	Backup import (fresh random ID, avoids clashing with the exporter's cloud rows)	`i17163840001234_0`
s	Strava import (Strava's own activity ID — also the dedup key)	`s12345678901`

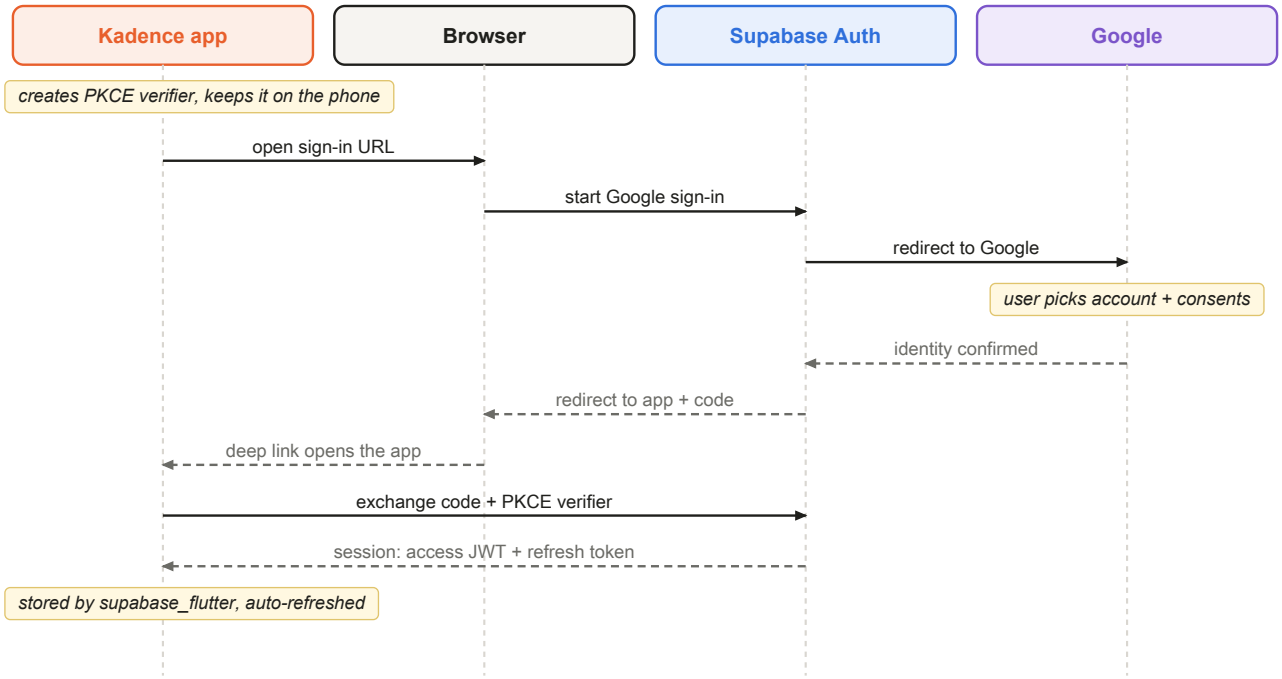
3. Authentication: Google + Supabase

Signing in answers one question: **which Kadence user is this?** Kadence never sees a password. Google proves the identity, Supabase turns that into its own session, and the app holds that session.

> **Sign-in is optional and Pro-only.** The only things an account unlocks are cloud sync and Strava — both Pro features — so the sign-in buttons open the paywall for free users.

The sign-in flow (OAuth with PKCE)

OAuth is the standard "log in with Google" handshake. PKCE adds a one-time secret generated on the phone, so a stolen redirect link is useless to anyone else.



What the session is used for

- The **access JWT** is a signed token that says "this is user *X*", valid for about an hour. supabase_flutter attaches it automatically to every database request and every edge function call, and refreshes it in the background.
- **Row Level Security (RLS)** rules in Postgres read the user ID from that JWT and only allow access to that user's rows. This is what stops one user reading another user's activities, even though everyone talks to the same database.
- Edge functions call `auth.getUser()` with the JWT to learn which user is calling.

Deep links

Both Google sign-in and Strava finish by redirecting to the app's custom URL scheme `io.supabase.kadence://`. `main.dart` routes them by host: `login-callback` → Supabase, `strava-callback` → StravaService.

> **Why routing is manual:** Supabase's built-in link detector treats *any* link containing `?code=` as its own sign-in callback — including Strava's. So automatic detection is turned off (`detectSessionInUri: false`) and the app decides. Re-test Google sign-in after upgrading supabase_flutter.

Sign-out and Pro expiry

Signing out ends the session but **keeps all local data**. If a Pro subscription lapses, ProController signs the user out (at start-up and on downgrade), which stops every cloud and Strava call — no extra checks are needed in the sync code.

4. Cloud sync

The phone stays the source of truth; Supabase holds a copy so data survives a lost phone and appears on a second device.

- **While signed in:** every save, check-off and delete is pushed to the `activities` table immediately (fire-and-forget, so offline is fine).
- **While signed out:** deletes are queued and sent at the next sign-in, so a deleted activity doesn't come back from the cloud.
- **On sign-in** a full sync runs. Which kind depends on whose data is on the phone (tracked by an "ownership tag"):

Situation	What happens
Nobody's data yet (first sign-in, or after a backup import)	Push everything up, pull cloud, merge
Same user as last time	Send queued deletes, pull cloud, merge, push local-only rows
A different user	Wipe the phone and load the new user's cloud data (the old data belonged to someone else)

Merge rule — last write wins: rows are matched by ID; when both sides have the same ID the cloud version wins (the database stamps `updated_at` on every change). Rows that only exist on one side are kept.

> **Current gap:** the app only *pulls* from the cloud at sign-in. Changes made on another device appear after the next sign-in, not live. Fixing this (pull on resume, or Supabase Realtime) is also a prerequisite for Strava webhooks.

Cloud tables

Table	Contents	Who can access
profiles	Name + avatar, created automatically on first sign-in	The user, own row only (RLS)
activities	Mirror of the local table + user_id + timestamps	The user, own rows only (RLS)
strava_tokens	Strava access/refresh tokens per user	**Nobody from the app.** Only edge functions (service role)

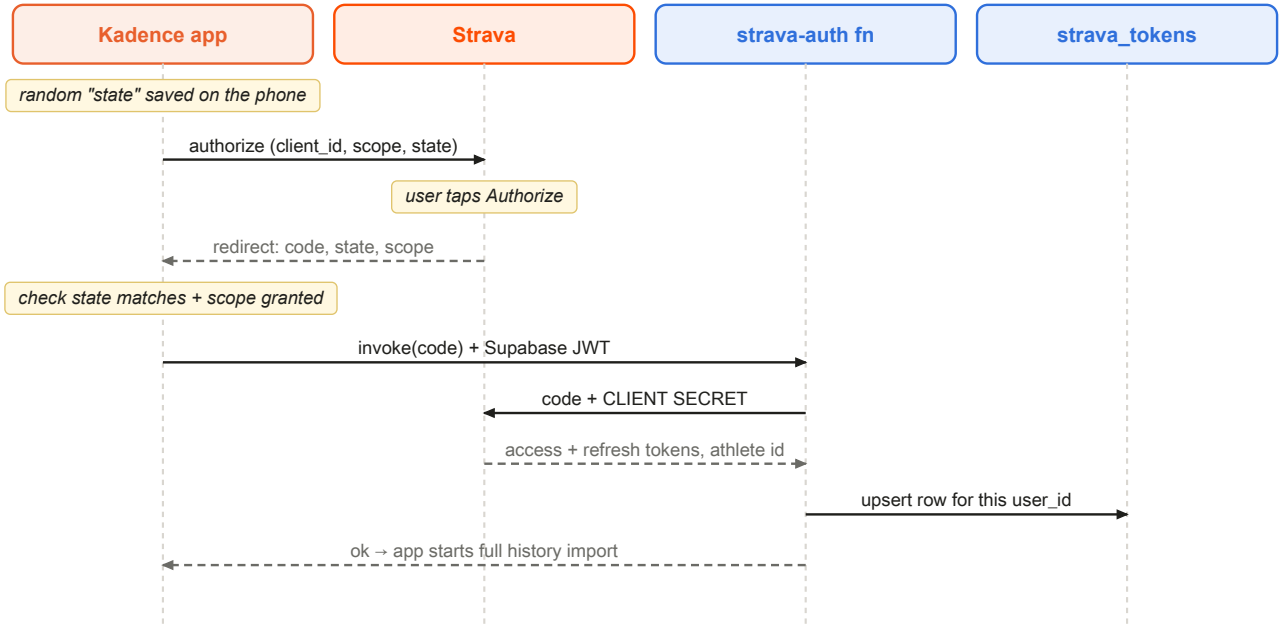
5. Strava integration

Strava access is a **second, separate OAuth**. Signing in to Kadence says who the user is; connecting Strava gives Kadence permission to read that person's Strava activities. It is one-way: Kadence reads from Strava and never writes back.

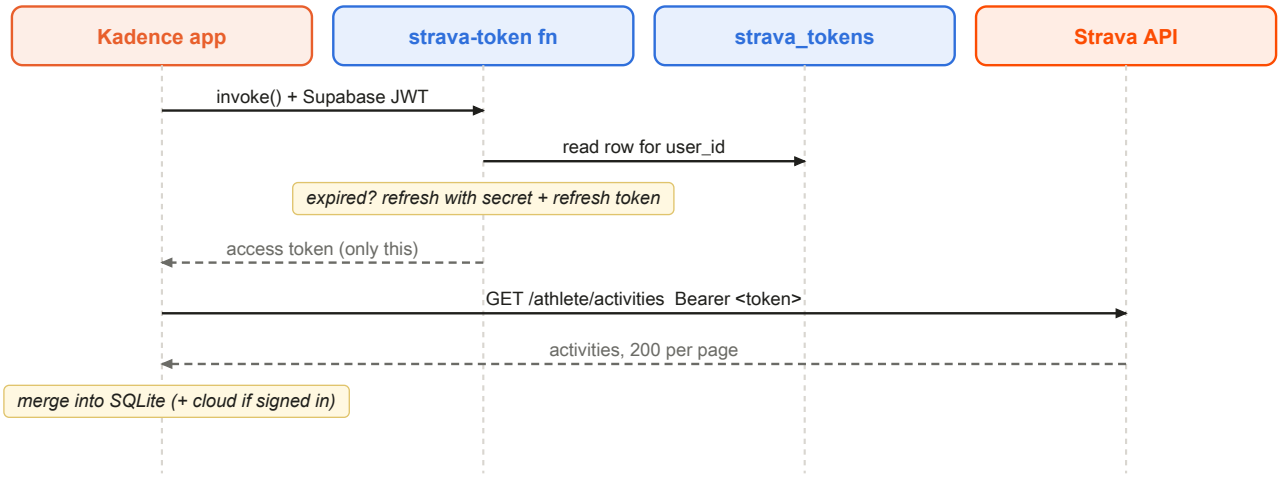
The credentials, and where each one lives

Credential	What it is	Lives in	Secret?
Client ID	Kadence's public app number at Strava	.env → built into the app	No — it appears in the authorize URL anyway
Client Secret	Kadence's app password at Strava	Supabase function secrets **only**	**Yes.** Anything shipped in the app can be extracted
Authorization code	One-time ticket returned after the user clicks "Authorize"	Passes through the phone once	Short-lived; useless without the secret
Access token	The *Bearer* token sent on Strava API calls. Expires after ~6 hours	strava_tokens; handed to the phone on request	Low risk — expires quickly
Refresh token	Long-lived key that mints new access tokens	strava_tokens **only**	**Yes.** Never leaves the server
Supabase JWT	Proves which Kadence user is calling the edge functions	Phone (supabase_flutter)	Per-user session, ~1 h

Connecting



Syncing



When syncs happen

Trigger	Range fetched
Right after connecting	Entire history (pages until empty)
App start-up and resume (if last sync > 15 min ago)	Since last sync minus 30 days — catches activities uploaded late
"Sync now" button	Same as above

Merge rules (additive — Strava never deletes anything)

- Already imported (ID `s<stravaId>` exists, or the Strava ID already ticked off a planned session) → **skip**.
- A planned, not-done session of the same type on that day → **mark it done**.
- Otherwise → **add** a done activity with Strava's title, start time and moving time.
- All changes go through one batched write: no calendar events and no review prompt per imported activity.

Disconnect vs sign-out

Action	Effect on Strava link	Imported activities
Strava → Disconnect	`strava-disconnect` revokes access on Strava and deletes the token row	Kept
Sign out of Kadence	Untouched on the server; paused until the same user signs in again	Kept
Sign in as another user	Shows "not connected" (the link belongs to the other account)	Replaced by that user's data

Before launch: new Strava API apps allow only 1 connected athlete (*Single Player Mode*). Every other user is blocked until Strava reviews the app. Reviews favour apps that use **webhooks** (Strava calls a Kadence server function when an activity is uploaded) instead of polling — planned as the next step, with the history import still done once through the API.

6. Secrets and configuration

Value	Where	In git?	Notes
SUPABASE_URL, SUPABASE_ANON_KEY	.env → --dart-define-from-file	No	Public by design; RLS is what protects data
STRAVA_CLIENT_ID	.env	No	Public by design
STRAVA_CLIENT_SECRET	Supabase function secrets	No	Never in the app
SUPABASE_SERVICE_ROLE_KEY	Injected into edge functions by Supabase	No	Bypasses RLS — must never reach the app
Android upload keystore	~/upload-keystore.jks + android/key.properties	No	Losing it complicates Play updates — back it up

Rule of thumb: the anon key and client IDs identify the app, they don't protect anything. Protection comes from RLS (database) and from keeping the client secret, refresh tokens and service-role key on the server.

7. Other integrations

Feature	How it works
Device calendar	Optional. Each activity becomes a calendar event (start from its time, default 08:00). The app is the source of truth: editing or deleting an activity updates or removes the event. Event IDs are stored on the activity.
Backup export / import	JSON file via the system share sheet. Import **replaces** all data, gives every activity a fresh ID, and (if signed in) replaces the user's cloud rows too.
In-app review	Native store dialog after 5+ days and 3+ activities, with growing cooldowns; the OS adds its own limits.
Tutorials	One-time gesture overlays (swipe, double-tap, long-press, title tap), remembered in SharedPreferences.

8. Pro gating

Pro is currently a local flag (the paywall purchase buttons are stubs). Gating happens only at UI entry points:

- Stats: everything below "Pro Stats" is blurred behind a lock.
- Settings: Theme color, Type colors, account sign-in and Strava show a PRO badge and open the paywall.
- Onboarding step 3: the cloud sign-in buttons are replaced by a Pro button.

Because every account feature requires sign-in, and non-Pro users are signed out automatically, the sync and Strava code don't need their own Pro checks.

9. Open items

Area	Next step
Purchases	Wire RevenueCat or in_app_purchase so ProController reflects real store entitlements
Strava	Deploy functions + migrations; webhooks; request athlete capacity; official "Connect with Strava" button asset
Cloud sync	Pull changes on app resume (or Realtime) so other devices' and webhook changes appear without re-signing in
Auth	Apple Sign-In (needs Apple Developer enrollment); native Google one-tap
Hosting	Supabase free projects pause after ~7 days idle and have no backups — move to Pro ($25/mo) before paid features go public